Not Approved. This proposal was explored using Claud. There are many wrong info and explanation in regards to real-life scenario. This is an idea only and it is better to just take a bit of inspiration from here and not follow it.

Project Proposal: On-Chip KV Reuse for FlashAttention Decoding

Prepared by Sumaiya Tarannum Noor; ID – 2425410650 · October 2026 · Draft 1

1. Summary

Autoregressive decoding with FlashAttention is memory-bound. With one query row, every cached K/V byte is read from HBM once per step and used once, so arithmetic intensity is about 1 FLOP/byte in fp16. We can study how much decode speed I can recover by making each K/V tile loaded into SRAM serve more query rows, and by shrinking the bytes read through a compressed "hot window" of the KV cache. We can build an analytical IO model, implement a decode kernel, and measure latency, HBM traffic and accuracy against standard baselines.

2. Background and Problem

The KV cache lives in HBM, not SRAM. For Llama-3-8B (32 layers, 8 KV heads, head dim 128, fp16) it costs 128 KiB per token, about 1 GiB at 8k context. An H100 SM has about 228 KB of shared memory, roughly 30 MB across the chip, so the whole cache can never be on-chip.

FlashAttention handles this by streaming K and V tiles from HBM through SRAM and combining partial results with an online softmax. For each query tile of B_r rows, the full cache is streamed once, so HBM traffic for the cache is about ceil(n_q / B_r) * 2 * t * d * b bytes, where t is cached tokens, d is head dim and b is bytes per element. Each cached byte is therefore reused B_r times per load.

In decode, n_q = 1 and B_r = 1, so there is no reuse. Decode latency is approximately cache bytes divided by HBM bandwidth.

3. Key Observation: SRAM Does Not Persist

Decode launches one kernel per layer per token, and shared memory is cleared when a kernel ends. A window of KV placed in SRAM is reloaded from HBM at every step, so "keeping KV in SRAM" by itself saves no HBM traffic. A speedup has to come from one of three sources:

1. **Reuse within one launch:** several query rows share each loaded tile (GQA groups, speculative decoding, batched requests with a shared prefix).

2. **Reading fewer bytes:** a quantized and/or smaller hot window means fewer bytes per step.
3. **Persistent kernels:** a megakernel that never exits across steps, feasible only for very small windows (roughly 240 tokens in fp16 across all layers on an H100, about 1,000 in int4).

This proposal tests (1) first because it is exact, then studies (2) as a separate accuracy-versus-speed question.

4. Research Questions

- **RQ1:** How well does the IO model, with arithmetic intensity about 2 * B_r / b, predict measured decode latency as B_r, context length and group size G vary?
- **RQ2:** What speedup does exact tile reuse give with GQA grouping, speculative decoding and shared-prefix batching?
- **RQ3:** For a sink-plus-recent window with int4/int8 quantization, what is the trade-off between tokens read, accuracy loss and decode latency?
- **RQ4:** What are the optimal tile size B_c and number of KV splits for a given SRAM size and bandwidth?

5. Proposed Approach

5.1 Analytical model

We can write HBM traffic and latency as a function of SRAM size M, tile sizes (B_r, B_c), KV splits S, bytes per element b, and group size G. The SRAM constraint is (2 * B_r * d + 2 * B_c * d + B_r * B_c) * b <= M. On-chip token capacity per layer is about M / (2 * H_kv * d * b_eff).

5.2 Exact tile-reuse kernel

We can implement a FlashAttention-style decode kernel (Triton first, CUDA if needed) that packs G query heads, or several speculative or batched queries, as rows against one KV tile. It uses the online-softmax update on (m, l, O) and split-KV merging, which is valid because the combine operator is associative.

5.3 Compressed hot-window variant

We can keep attention-sink tokens plus the most recent N tokens (and optionally a top-k set) in a low-bit quantized form, and attend over the window and the remaining cache using the same (m, l, O) merge. We can report window-only and window-plus-full-cache modes separately, since only the second is exact up to quantization error.

6. Evaluation Plan

- **Models:** one open GQA model (for example Llama-3-8B) at several context lengths.
- **Baselines:** fp16 FlashAttention decode, FlashDecoding (split-KV), GQA-only, and quantization-only.
- **Metrics:** decode latency per token, HBM bytes moved (Nsight Compute), SM and DRAM throughput utilization, perplexity and a long-context benchmark for accuracy.
- **Sweeps:** B_r, G, context length, bit width, window size, number of KV splits.
- **Model validation:** plot measured latency against the model's prediction and report the error.

7. Milestones

1. **Phase 1 (weeks 1-2):** derive the IO model and set up the benchmark harness and baselines.
2. **Phase 2 (weeks 3-4):** implement the exact tile-reuse kernel and validate correctness against reference attention.
3. **Phase 3 (weeks 5-6):** profile with Nsight Compute and compare to the model (RQ1, RQ2, RQ4).
4. **Phase 4 (weeks 7-8):** implement the quantized hot-window variant and run accuracy and speed studies (RQ3).
5. **Phase 5 (weeks 9-10):** write up results.

8. Risks and Mitigations

- **Little speedup beyond FlashDecoding or GQA:** We can report it as a finding and focus on where the model predicts gains (small G, long context).
- **Accuracy loss from windowing or quantization:** We can measure it explicitly and keep the full-cache mode as the exact reference.
- **Kernel engineering effort:** We can start in Triton and move to CUDA only if profiling shows a clear need.
- **Hardware dependence:** We can state results per GPU, since the ridge point and SRAM size differ across A100 and H100.

9. Expected Contributions

1. An IO model linking SRAM size, reuse factor, quantization width and window size to decode HBM traffic and latency.
2. A decode kernel that merges SRAM-resident tiles with streamed HBM tiles using the online-softmax combine.
3. A measured comparison against fp16 FlashDecoding, GQA-only and quantization-only baselines, with an honest account of where on-chip reuse helps and where it does not.

10. Claims Policy

We can describe only the hot window as "kept in SRAM", never the whole KV cache. We can state speedup and accuracy claims only after measurement; until then the proposal says "the model predicts" or "I expect".